

Lecture 2

Software Engineering Processes

Course 2026/2027



Outline

1. What are Software Processes
2. Generic Process Models
   a. Waterfall
   b. Rapid Application Development
   c. V model
   d. Prototyping
   e. Spiral
   f. Other
3. Agile Methodology

HOW MANY PROGRAMMERS DOES IT TAKE TO CHANGE A LIGHT BULB?



Processes

What is a software process?

The set of activities and associated results which produce a software product or System

(in other words, the sequence of steps required to develop and maintain software)

- sets out the technical and management framework for applying methods, tools and people to the software task
- guides software engineers as they work by identifying their roles and tasks

What are the common activities?

1. Feasibility/marketing study (optional)
   - Feasibility report
2. Software Specification
   - Define functionality and constraints
3. Software Development
   - Produce Software
4. Software Verification and Validation
   - Does what the customer wants
   - Matches the specification
5. Software Maintenance and Evolution
   - To meet customer changing needs

1. Feasibility Study

The goal is to decide if the proposed system is worthwhile

The Feasibility Study is a short, focused study, that checks:

- If the system contributes to organizational objectives
- If the system can be engineered using current technology and within budget
- If the system can be integrated with other systems that are used

2. Software Specification (aka Requirements Engineering)

Understand system's requirements (functional and non-functional)

Leads to requirements document:

- get high-level statements from stakeholders (users, administrators, legal entities, etc)
- get detailed system specification

3. Software development (design and implementation)

Develop an executable system for delivery to the customer

Design:

- **Architectural Design:** identify overall structure of the system (components, relations and distribution)
- **Interface Design:** Interfaces between components
- **Component Selection and Design:** look for reusable components or design new ones (can leave details to programmer)
- **Database Design:** Data Structures Design

Implementation:

You already have "an idea"...

4. Verification and validation

Verification:

Are we building the system right?

(according to the specifications)

Validation:

Are we building the right system?

(meeting the customers expectations)

5. Software Maintenance and Evolution

Software is continuously changing over its lifetime due to changes in requirements, customer needs and other context changes.

- Corrective
- Adaptive
- Perfective

6. System Delivery & Operation

Implementation of the system into the working environment and replacement of the existing system

Operation support

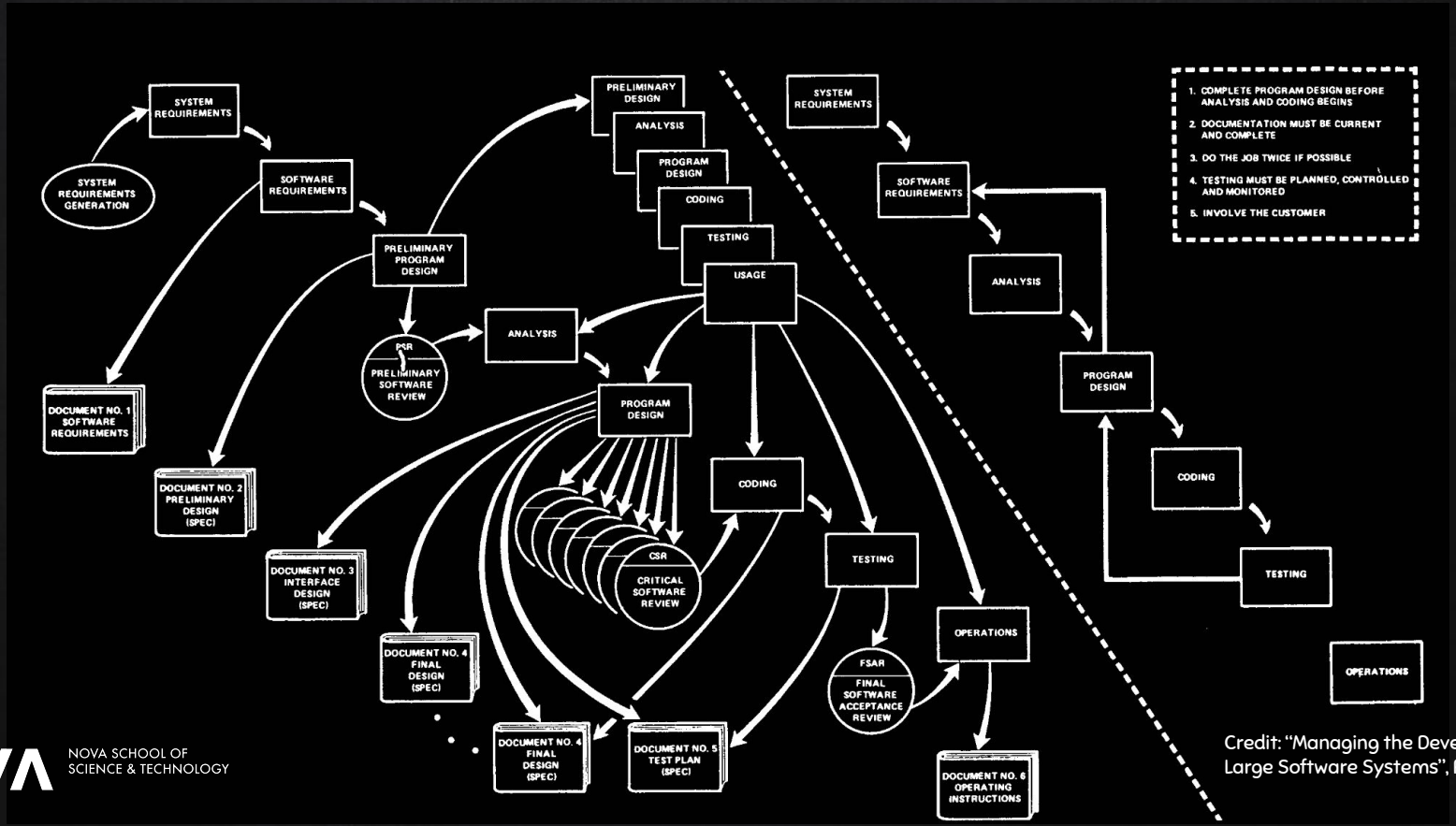


Generic Processes

There is NO UNIVERSAL PROCESS!



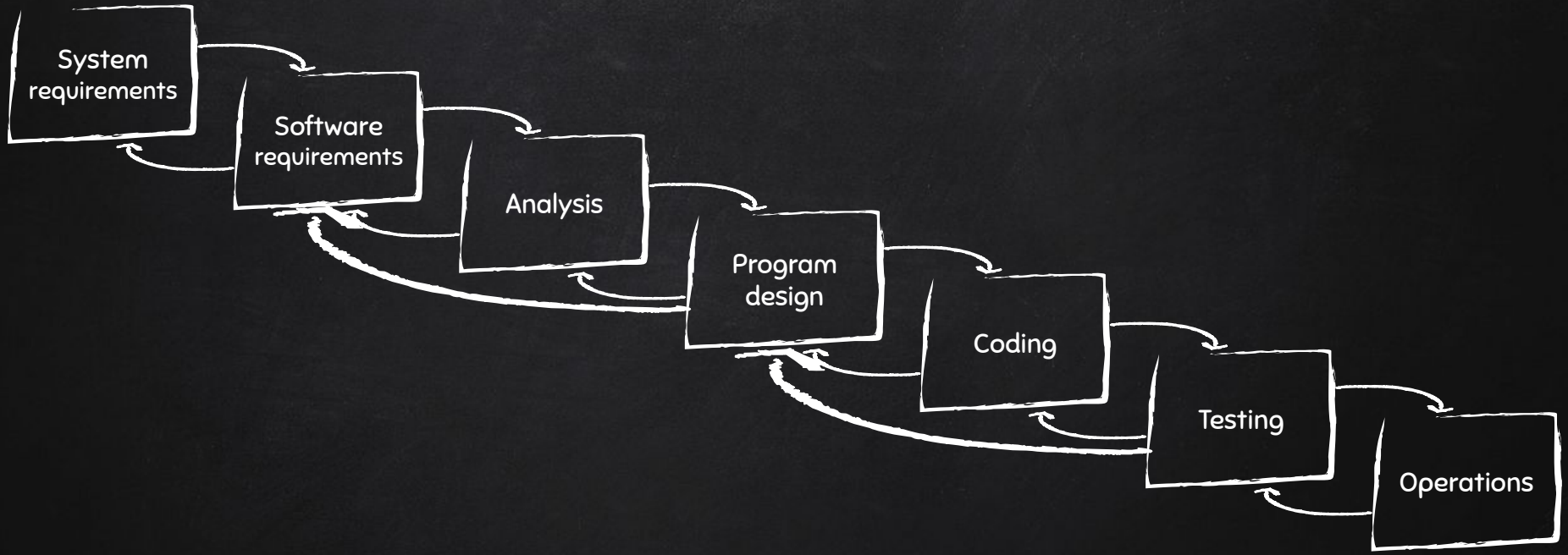
Waterfall model
(Royce 1970)



Credit: "Managing the Development of Large Software Systems", Royce, 1970

Waterfall Model

(Abstracted from Royce 1970)



Each phase results in deliverables including documentation
The following phase should not start until the previous has finished

Waterfall

Each phase results in deliverables including documentation
The following phase should not start until the previous has finished

Advantages

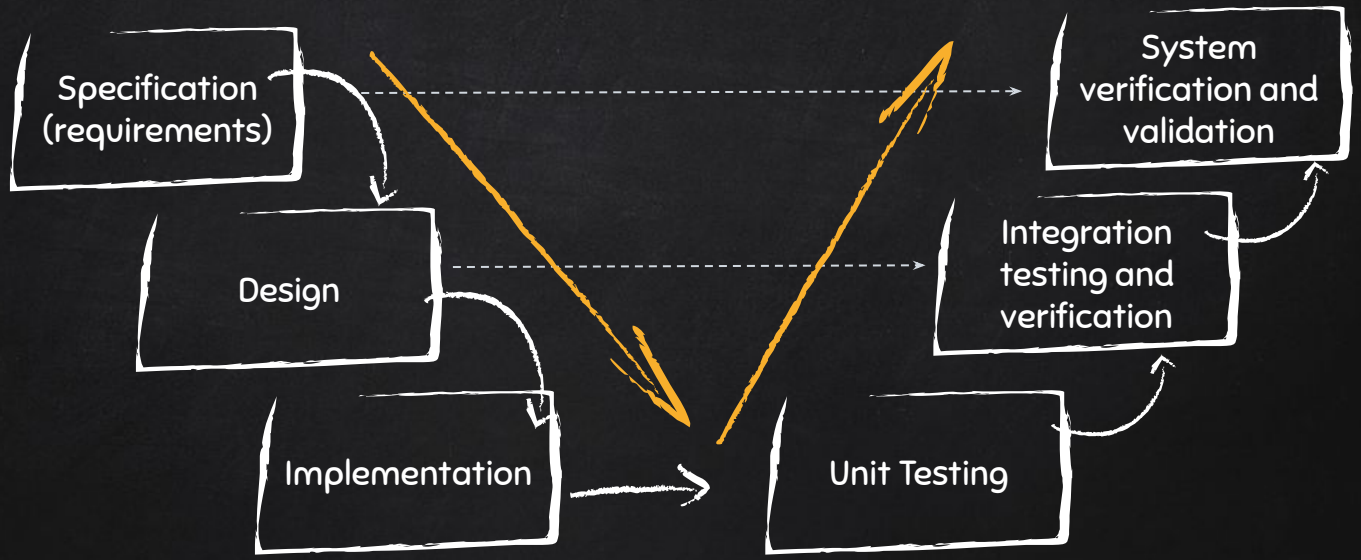
- Used in other engineering approaches
- Is visible so managers can monitor the process against the plan
- Perform better when the requirements are well understood and the requirements unlikely change during the system development
- Adequate to large projects with large multidisciplinary teams or subsystem integration needs
- Suitable for well-understood developments using familiar technology

Disadvantages

- Inflexible partitioning of the project into stages
- Costs of iteration lead to costly documentation and involve significant rework
- The customer only sees the system at the late phases where changes are difficult
- Commitments must be made at an early stage in the process, which makes it difficult to respond to change

V model

Adaptation of the waterfall used in Systems Engineering with focus on tests. The tests are designed in parallel with each phase of the life-cycle. Often used in safety-critical systems, such as aerospace and defense systems, because of its emphasis on thorough testing and its ability to clearly define the steps involved in the software development process.



V model

Advantages

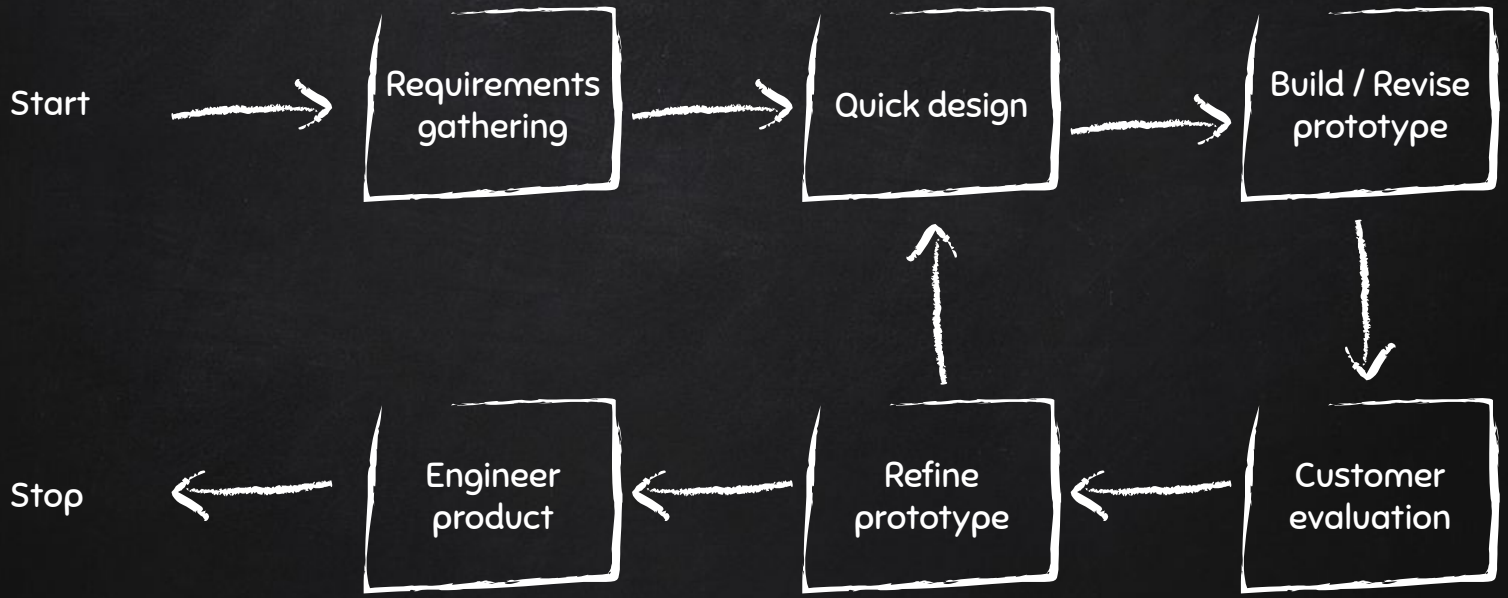
- Minimizes project risks – due to the explicit concerns:
  - Verification
    (Am I doing things right?)
  - Validation
    (Am I doing the right thing?)
- Improvement and guarantee of quality
- Reduction of total cost over the entire project and system's life cycle

Disadvantages

- Apart from the mentioned benefits it suffers from the same problems of the waterfall model

Prototyping (Disposable)

Small replicas of the future applications built before a full application.
They are used to be built in a hurry.



Prototyping

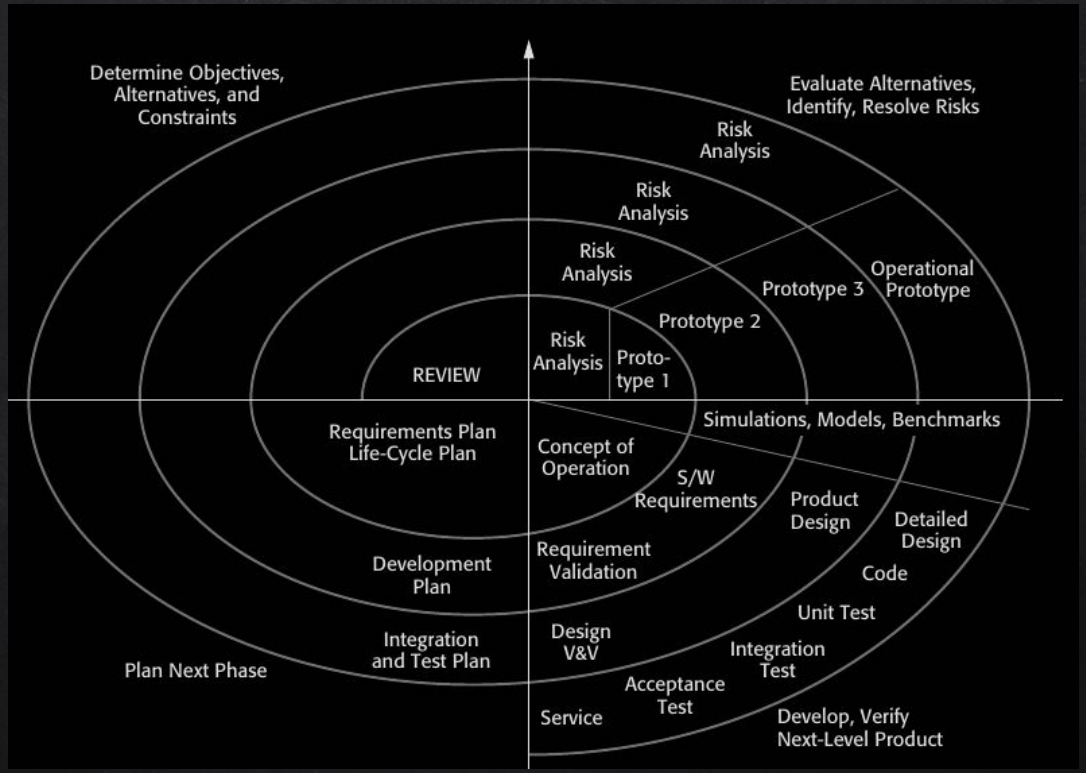
Advantages

- Ideally mock-up serves as mechanism for identifying requirements and can be integrated in other processes
- Users like the method, get a feeling for the actual system
- Less ideally may be the basis for completed product
- Adequate for studying technological feasibility or learning new technology when the team is unexperienced with them

Disadvantages

- prototypes often ignore quality/performance/maintenance issues
- No stable architecture, and poorly structured
- Not cost effective to produce documentation
- may create pressure from users to deliver earlier
- may use a less-than-ideal platform to deliver
- Not adequate for large, complex long-lived systems with different teams developing different parts

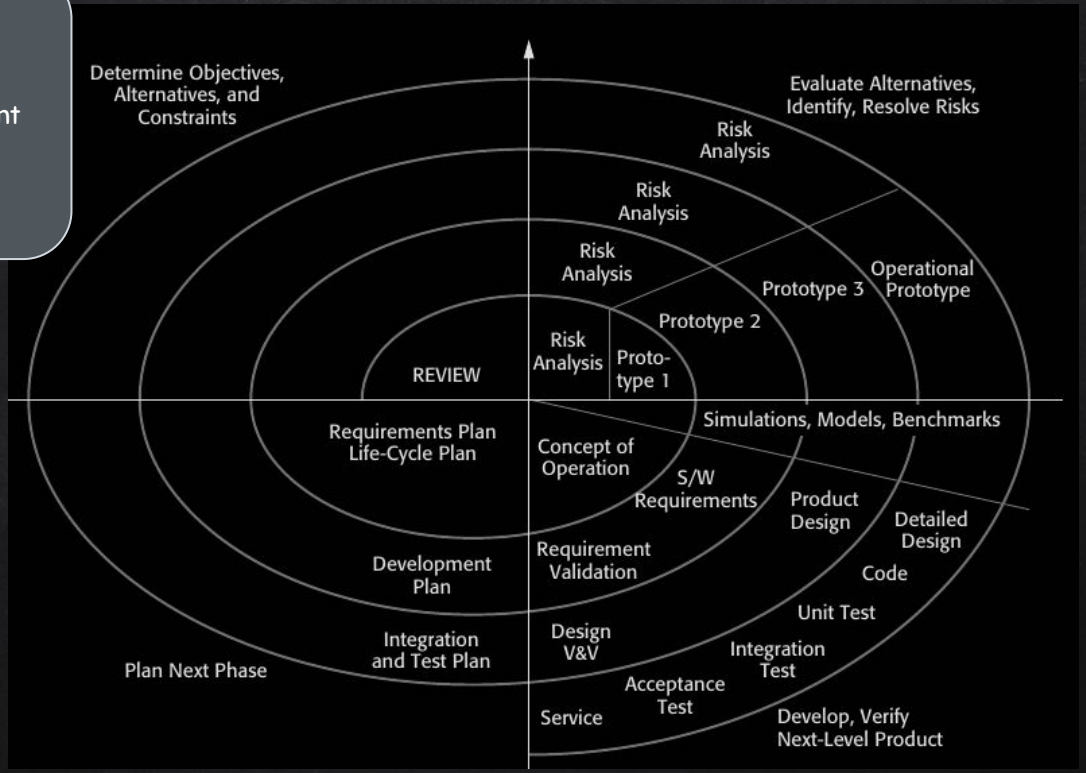
Spiral Model
(Bohem 1988)



Credit: "Software Engineering", Ian Sommerville 9th edition

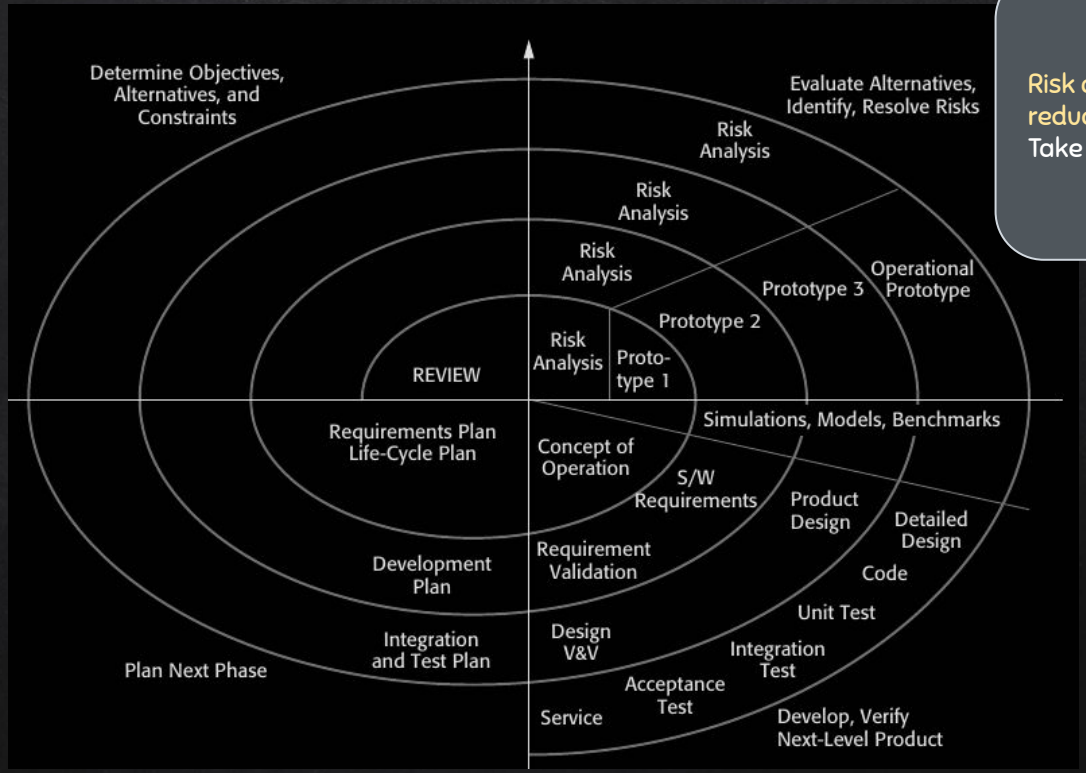
Spiral Model
(Bohem 1988)

Objective Setting
Define objectives
Define detailed management plan
Identify constraints



Credit: "Software Engineering", Ian Sommerville 9th edition

Spiral Model
(Bohem 1988)

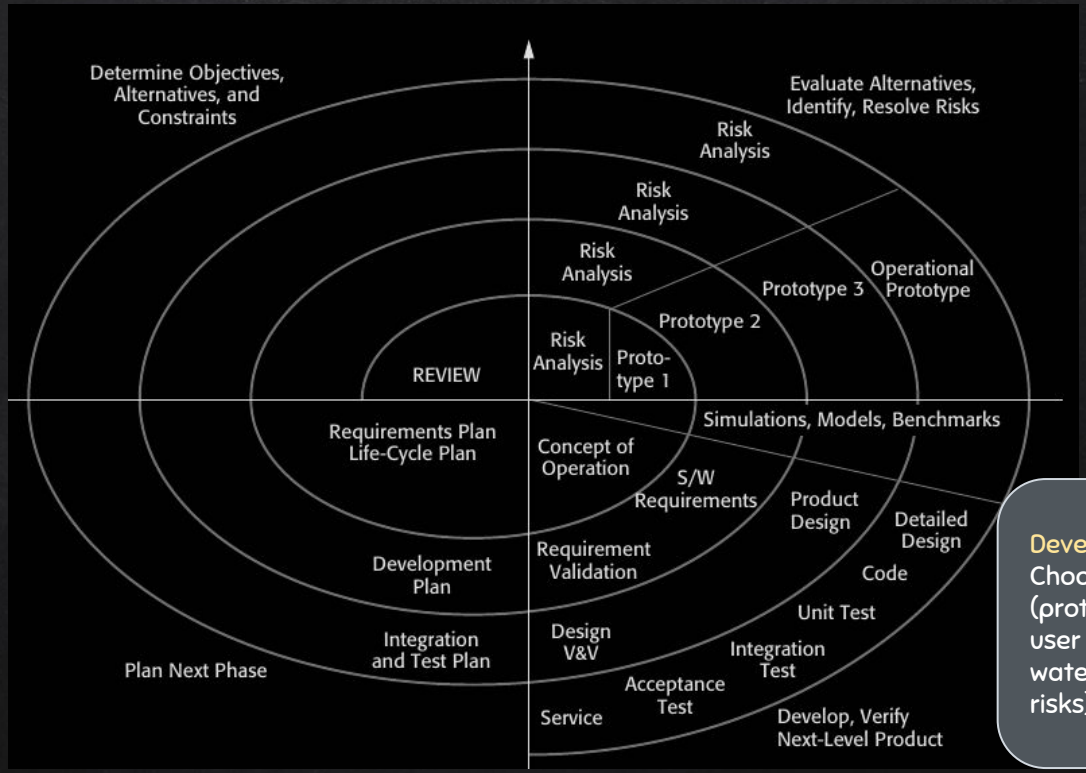


Risk assessment and reduction

Take steps to reduce risk

Credit: "Software Engineering", Ian Sommerville 9th edition

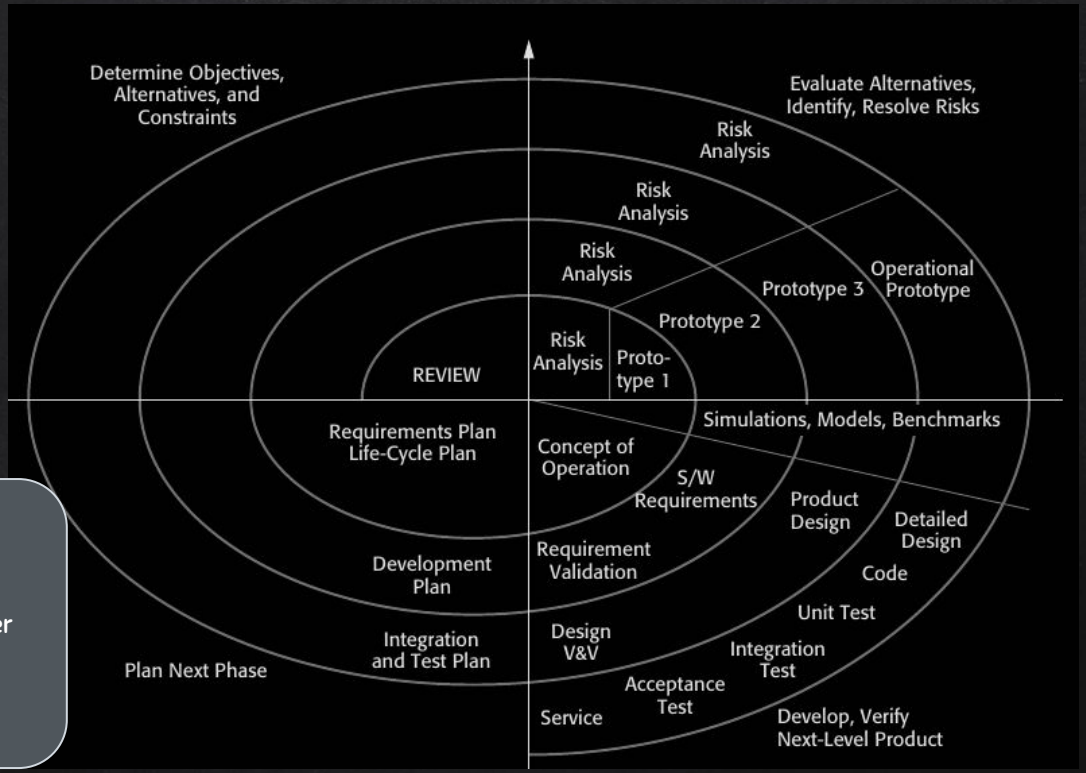
Spiral Model
(Bohem 1988)



Development and Validation
Choose development model (prototyping for e.g. dominant user interface risks, or formal waterfall if e.g. there are safety risks)

Credit: "Software Engineering", Ian Sommerville 9th edition

Spiral Model
(Bohem 1988)



Planning
Review project
Decide go-no-go to a further loop.
Draw plans for next phase

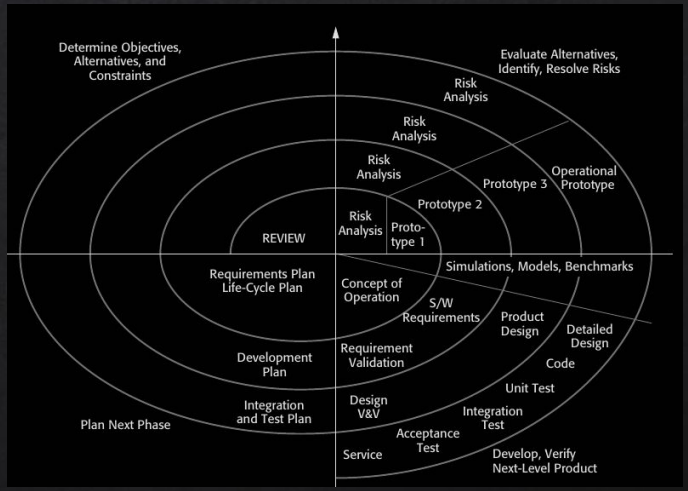
Credit: "Software Engineering", Ian Sommerville 9th edition

Spiral Model
(Bohem 1988)

Iterative nature (each iteration implies a delivery)

Meant for supporting the complete life-cycle of the system

High focus on risk analysis



Credit: "Software Engineering", Ian Sommerville 9th edition

Spiral Model
(Bohem 1988)

Advantages

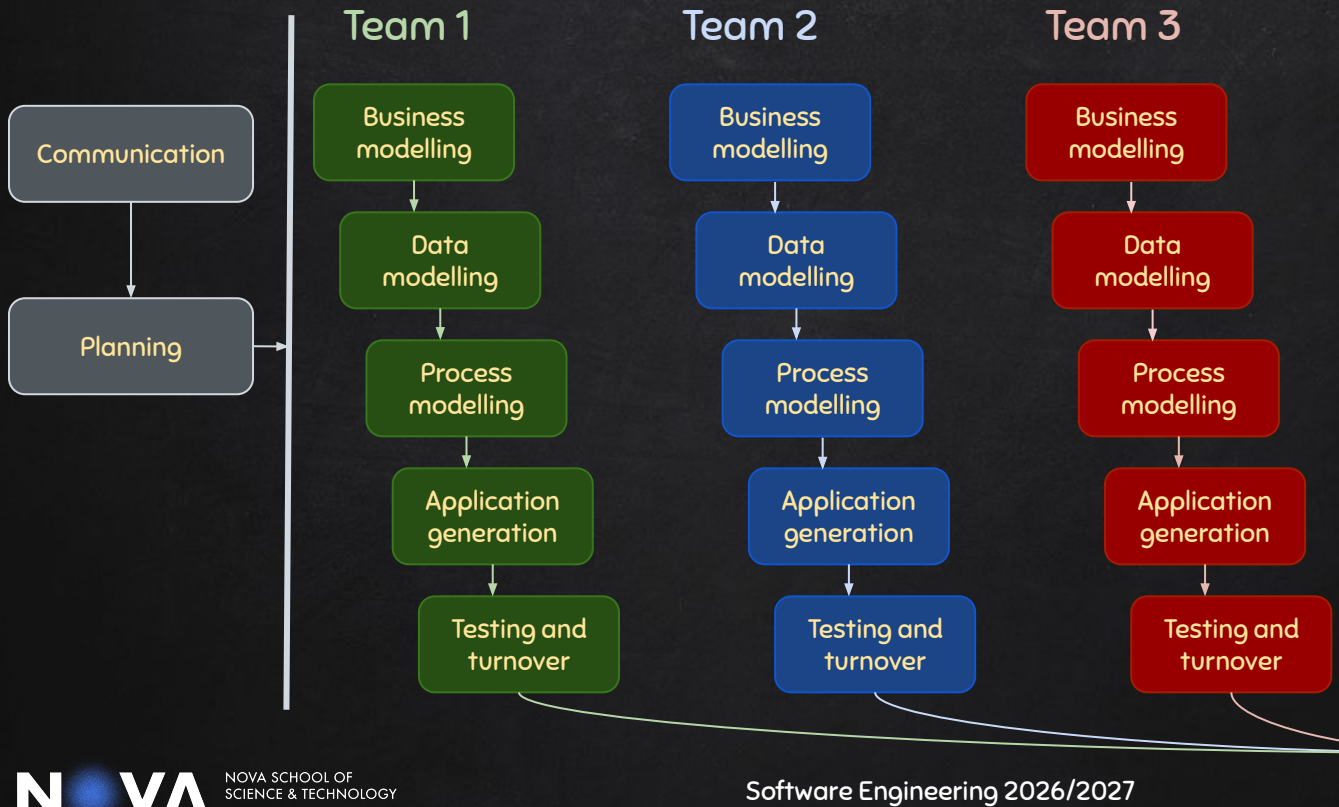
- From the traditional is considered one of the best approaches for its incrementality where the stakeholders are soon presented with versions until the system is developed completely
- Adequate for large scale software development
- Very strong in risk handling unanticipated risks
- Follows the complete software life-cycle until it is no longer used

Disadvantages

- Customer interaction/involvement is not at focus
- Requires excellent management and risk assessment

OTHER PROCESSES...

RAD – RAPID APPLICATION DEVELOPMENT



Similar problems to the waterfall

But Faster

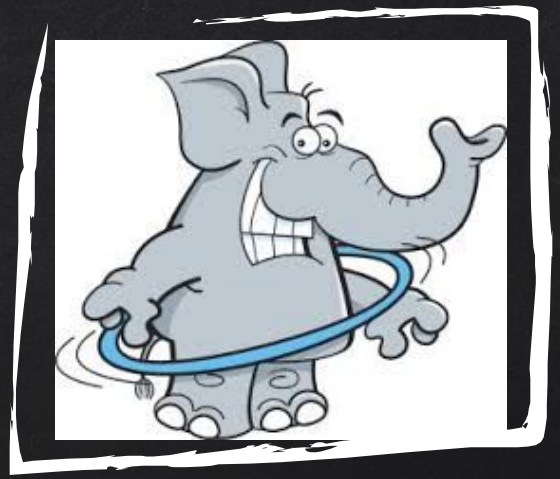
Requires large teams

Requires careful integration

Other processes...
Formal Systems Development Methods

- Such as B-method /Schneider 2001; Wordswoth 1996) or VDM (Vienna Development Method):
  - created from formal specifications (mathematical models)
  - there are tools to support model analysis and then automatic translation to code
- Suited for systems with stringent safety and reliability or security needs
- Mostly embedded systems
- Require very specialized expertise

Going Agile



Motivation for becoming agile

Previous plan-driven approaches

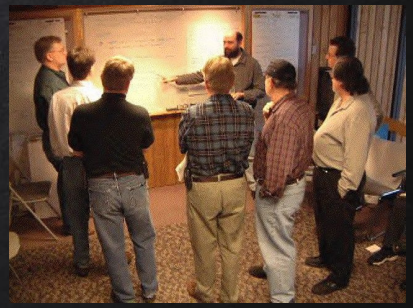
- developed for software development by large teams
- working for different companies
- often geographically dispersed
- working on software for long time
- e.g. software for: Automotive, Avionics

But, ...

- too heavy for small and medium business systems

more time spent on organizing, documenting and less on developing and testing

Agile Manifesto values



The goal is to uncover better ways of developing software.

- Individuals and Interactions more than processes and tools
  - process and tools help but persons are the key
- Working Software more than comprehensive documentation
  - documents are useful but we should focus "on the real value"
- Customer Collaboration more than contract negotiation
  - instead of distanced client that only shows in beginning and end and only looks for contractual compliance
- Responding to Change more than following a plan
  - being flexible and adaptive dealing with change with small cycles instead of following a rigid plan

That is, while there is value in the items on the right, the manifesto values the items on the left more.
Does not mean we should not do the things on the right!!!!!



Agile approaches focus on software and not on design and documentation (not plan driven).

Ideal for:

- small medium sized products and apps
- custom system development within organization

Specification, design and implementation are intertwined

Adequate when requirements change rapidly, meant to deliver working software quickly.

Agile Principles

Customer satisfaction by early and continuous delivery of valuable software

Welcome changing requirements even in late development

Deliver working software frequently weeks rather than months

Close, daily cooperation between customer and developers (provide and help prioritize new requirements)

Projects are built around **motivated individuals** who should be trusted

Face-to-face conversation is the best form of communication (co-location)

Working software is the primary measure of progress

Sustainable development, able to maintain a constant pace

Continuous attention to technical excellence and good design

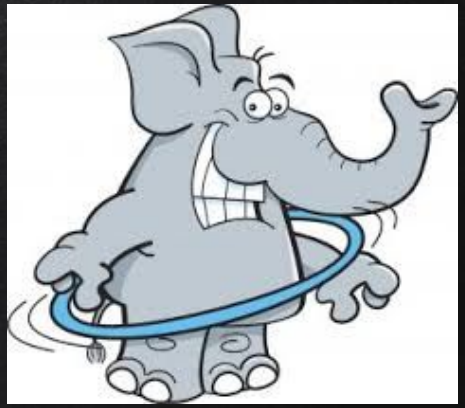
Simplicity—the art of maximizing the amount of work not done—is essential
(by deleting procedures that no longer apply, automating manual processes, using existing libraries rather than writing your own)

Best architectures, requirements, and designs emerge from **self-organizing teams**

Regularly, the **team reflects on how to become more effective**, and adjusts accordingly



Outline



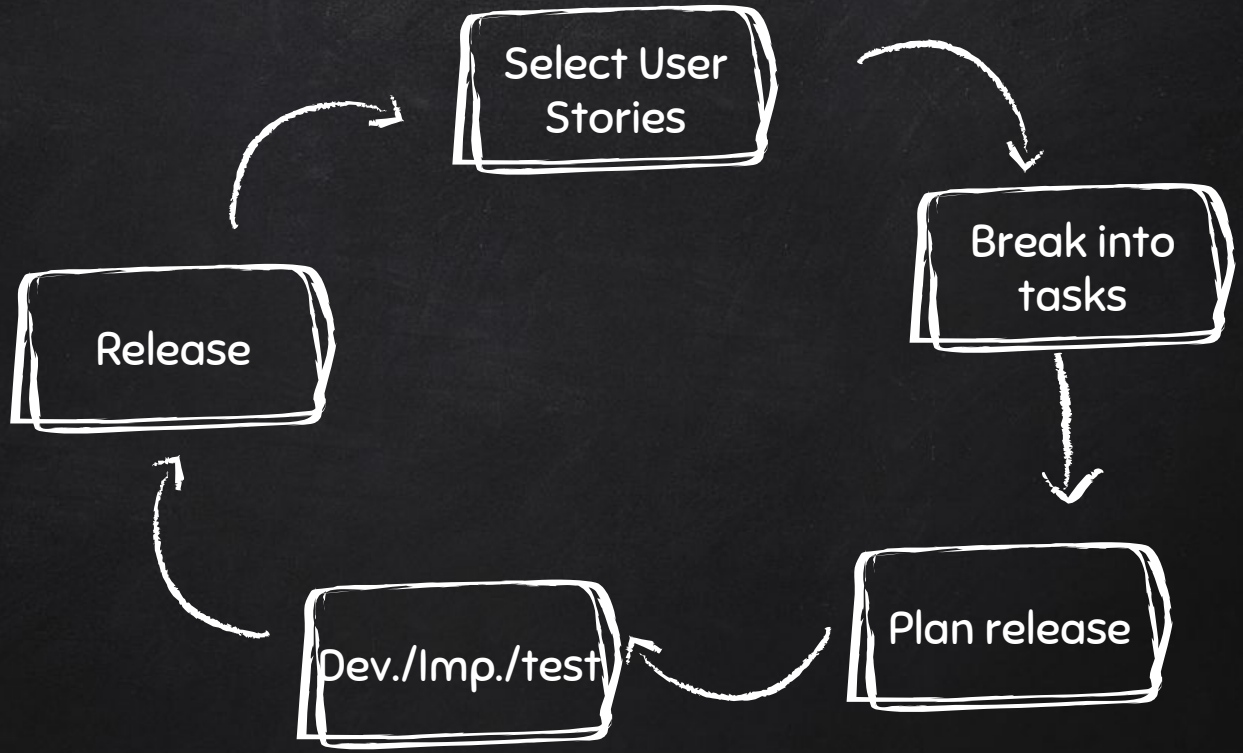
Agile Processes

- XP (Peer Programming, Test-Driven Development)
- Scrum

Industry use and issues

Agile Processes: XP

XP – eXtreme Programming (Kent Beck 1998)



XP – eXtreme Programming (Kent Beck 1998)

Loop of activities where (frequent iterations can happen several time per day, or up to 2 weeks):

1. We select a set of user stories (in story cards)
2. these are broken down into tasks
3. Plan release
4. Development/implementation/test (can happen intertwined)
5. release and then we evaluate it
6. Go back to 1.

User Stories



What is a user story?

A short and simple description, written in natural language, of a system feature, with value to the user or its owner.

Why would we use user stories?

The Agile paradigm promotes communication between different stakeholders, with diverse backgrounds and expertise

User stories are written in natural language and used to help identifying and prioritising requirements

Epic user stories

As a user, I want to backup my entire hard drive so that I can recover from accidental data losses.

This is a high-level user story, often called an epic.

You will want to break it down into more refined user stories.

Refined user stories

As a power user, I want to specify files or folders to backup based on file size, date created and date modified, so that I can filter what to backup.

As a user, I want to indicate folders not to backup so that my backup drive isn't filled up with things I don't need saved.

...

User stories templates

As a <type of user>, I want to <goal>, so that <benefit>.

As a user, I want to indicate folders not to backup so that my backup drive isn't filled up with things I don't need saved.

User stories templates can help mitigating the shortcomings of using natural language (e.g. ambiguity, tacit knowledge)

User stories templates Extended with Acceptance Criteria

As a <type of user>, I want to <goal>, so that <benefit>.

Acceptance Criteria:

Given <How things Begin>

When <Action taken>

Then <Outcome of taking the action>

User stories templates Extended with Acceptance Criteria

User Story

As a customer,

I want to select and purchase cinema seats online,

so that I can reserve my seats before arriving at the cinema.

Acceptance Criteria

Given a movie has available seats

When I select two available seats and confirm the purchase

Then those seats are reserved for me.

Persona



Persona: is a made up character with a certain profile that interacts directly or indirectly with the system in a story

User role vs. Persona

User roles = collection of defining attributes that characterise a population of users and their intended interactions

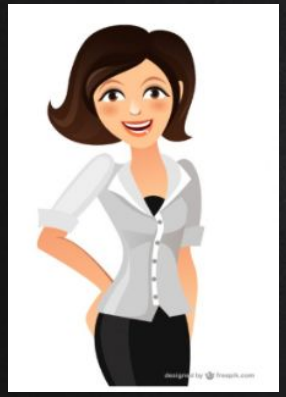
Persona = imaginary representation of a user role. Describe sufficiently so the team feel like they know the person (e.g. job, family, likes, comfort with technology). Includes a description and a picture.

Persona Description



- Name
- Representative photograph
- Quote that conveys what they most care about
- Job title
- Demographics
- Needs/goals
- Relevant motivations and attitudes
- Related tasks and behaviours
- Frustrations/ pain points with current solution
- Level of expertise/ knowledge (in the relevant domain, e.g. level computer savvy)
- Product usage context/ environment (e.g. laptop in a loud, bust office or tablet on the couch at home)
- Technology adoption life cycle segment (for your product category)

Persona Example





Vanessa is a 39 year-old resident of San Francisco. She is pursuing her passion for traveling after having a highly successful career as an attorney. She likes to have options while picking air travel and accommodation services so that she can choose the best and the most affordable. She gets frustrated with slow and cluttered websites.

User stories – shortcomings

Incompleteness

- Do all the stories cover the essential requirements?
- Is a story a true picture of an activity?



A lot of implicit knowledge might be ignored

Extreme Programming Summary

- Requirements as scenarios (user stories in story cards) implemented as tasks ranked by priority and time available
- Programmers in pairs, writing tests before code
- All tests must be successfully executed when integrating new code
- Incremental planning with short time between releases adding functionality

Agile Processes:

XP

Techniques:

TDD, Pair Programming, Simple Design, Sustainable Pace

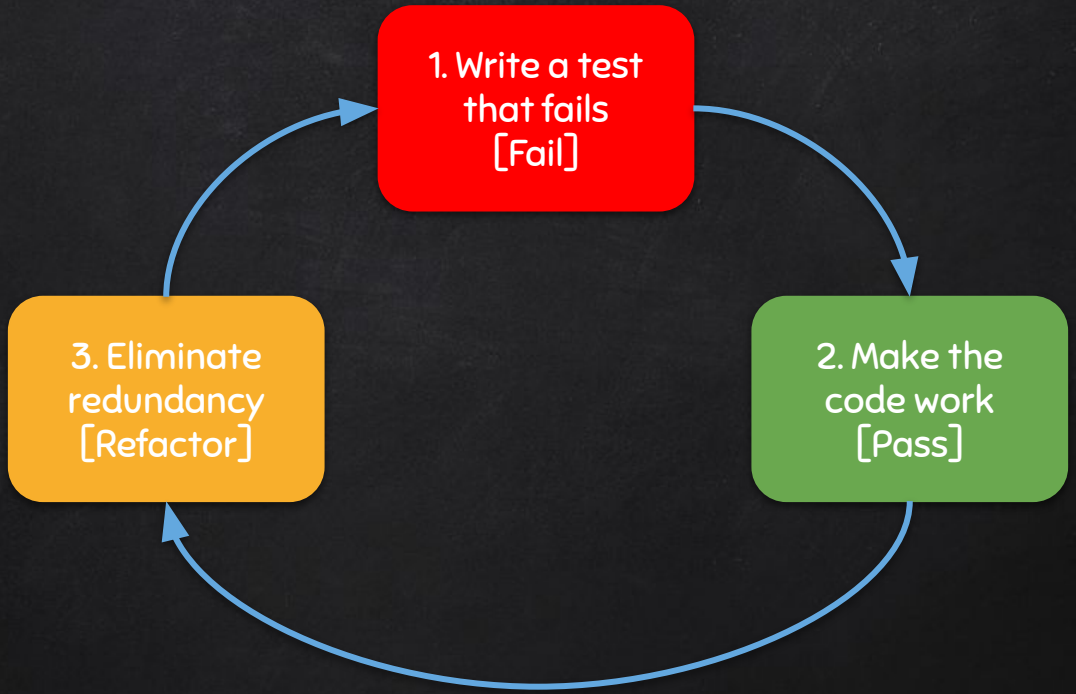
Test-Driven Development: test first implement later

As XP it is not plan driven, an external test team cannot base their work on the specification to build the tests

Therefore , you write your test before starting to develop (focus on what is needed)

Avoids the bias of building tests after having the knowledge of what was implemented

TDD: Test First life cycle



TDD Problems



In practice:

- Difficult to judge completeness
- Programmers tend to shortcut (they prefer code to tests)
- Some tests are difficult to write incrementally

Pair-Programming

Two developers program together:

- sharing the same computer monitor, keyboard etc
  - The Driver – writes while thinking out loud explaining what she/he is thinking and where she/he is going
  - The Navigator – comments
- Both share code ownership and responsibility
- Re-factoring is promoted

Pair programming Advantages

- Avoids bugs (two pairs of eyes are better than one for detecting problems in the code)
- Good for disseminating knowledge (sharing leads to more people knowing what each part of the code is doing)
- Good for onboarding people with no experience (to learn guidelines, etc)

Frequently used, nowadays!

XP techniques: Simple Design

Keep it simple, just solve the problem in hands!

- Planning for reuse is a waste of time
  - When writing code we typically think of all the possible use cases, predicting possible future changes and accommodate change, and try to make it generic (but most of the times it leads to overdoing)
- The idea of Simple Design is that overdoing is not a good idea, just solve the problem in hands with the knowledge at the time and don't think ahead
- However, code should be constantly refactored and immediately improved as new knowledge arrives

XP techniques: Sustainable pace

In the past, mostly in the 90's, there were the "programming death marches": a deadline was coming up and teams worked over time to get it done

This is not sustainable: it damages developers' physical and mental health, and decreases quality as the resources are exhausted

Instead, continuously work in a reasonable pace and amount of time. If you don't do it, it will come in the next release.

HOW MANY PROGRAMMERS DOES IT TAKE TO CHANGE A LIGHT BULB?

None, that's a hardware problem.



See you in the next Lecture



References

Software Engineering, by Ian Sommerville, Pearson, (either 9th or 10th edition)

see chapters 2 and 3

The Agile Manifesto: https://agilemanifesto.org/

